Assignment 2

This assignment is about implementing a Database Management System.

Template: assignment-2-project.zip

Description

You are tasked with creating a simple Database Management System (DBMS) for managing student records at a university.

Requirements

- Implement a data structure of your choice (either **B-Tree** or **Skip List**) to manage student records and the indices.
- Create a hash function to build an index based on the overall score (midterm + final).
- Develop another hash function to build an index based on the student number.
- An example DBMS class is provided which facilitates querying using your chosen data structure.
- A Student class is provided below to represent individual student records.
- Your implementation shall have good performance.
- You need to write at least *3* different test cases. **Do not test with large amount of data, or with multi-threading.**
- In case your function returns an object, and the object is not available (e.g., the function suppose to return a `Student` object, but such object does not exist) , you shall return `null` .
- In case your function returns a container (e.g., `List<Student>`), and no objects are available (e.g., the function suppose to return a `List<Student>` , but no `Student` available) , you shall return an empty container (e.g., `new ArrayList<>()`).
- You need to write Javadoc for the functions defined in `DBMSInterface` .
- When writing tests for a function returned multiple elements (e.g., `List<Student>`), you shall NOT check the order of the elements.
- DO NOT use `.hashCode()` , write your own hash function.

Submission

It is extremely important that you are following the naming conventions for your class, tests, files, and submissions. If the automatic system fails to recognise your submissions or failed to execute them, your assignment will be treated as NOT SUBMITTED. Late submissions are NOT allowed and will NOT be accepted.

Requirements

- Your submission must be a zip file and the file name must be your student number. Example: `202312345.zip`
- Your submitted zip file must contain `DBMS.java` and `DBMSTest<student number>.java` . The structure shall like:

```
202312345.zip
    |- DBMS.java
    |- DBMSTest000000000.java
```

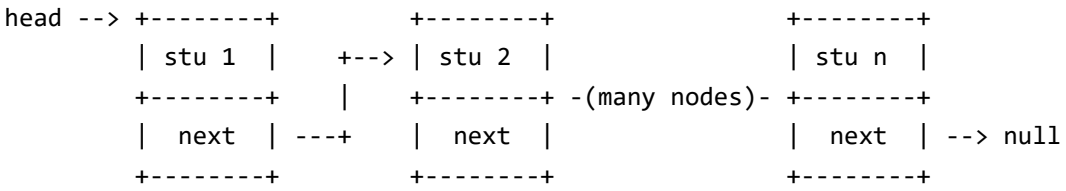
- The `DBMS.java` file must include a public class, `DBMS`, which implements the `DBMSInterface` interface.
- You can change anything in your submission.
- No third-party libraries or dependencies.
- Your code must be working with JDK 17 and up.
- The `App.java` file include a basic example. Your program shall be, at minimal, working with the example.
- All your own tests shall pass.
- Code style is important. You need to follow Java naming conventions and ensure there are no unnecessary extra new lines, spaces (e.g., additional spaces at the end of lines, a line with only spaces), etc.
- Do not declare packages. You must use the default package.
- You can use JDK provided LinkedList if you wish
- When inserting a new `Student` object with the same student number, you shall replace the old `Student` object with the new one.

Hints (you don't have to read or follow this section)

- Making a hash function work is easy, but designing a good hash function is difficult.
- When in doubt, debug your code line by line to understand how things work.

Some suggestions regarding the architecture design.

- Design a storage pool to store all students, such as a linked list.



- Implement the `DBMSInterface` in your `DBMS` class with naive implementation, e.g., by searching the linked list one by one. Your naive solution shall work perfectly fine, although it has to go through every node in the linked list when we search
- Design hash functions to convert Student to hash code (integer). We need two hash functions to support building indices for both Student Number and Score.
- Construct B-Tree or Skip List to work on the hash code you generated. The B-Tree or Skip List shall store the hash code and pointers to your Student object in the storage pool.
- Update your `DBMS` class to use B-Tree or Skip List to search for Student object. The search shall be much faster than the naive implementation.
- You don't have to use `StorageBackendInterface`. If you choose to use it and have questions regarding how to deal with deleting students with score, here's an illustration on what you can do:

```java
// In DBMS class
@Override
public void deleteStudent(Student student) {
    this.studentNumberIndex.delete(student.getStudentNumber());

    // retrieve all students with that score
    List<Student> students = this.overallScoreIndex.get(student.getOverallScore());

    // remove the student you want to delete
    students.remove(student);

    // remove every student with that score from the index
    this.overallScoreIndex.delete(student.getOverallScore());

    // re-insert students with that score back to the index
    for (Student s : students) {
        this.overallScoreIndex.insert(s.getOverallScore(), s);
    }
}
```